

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET
DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ D'ABOMEY-CALAVI
FACULTÉ DES SCIENCES ET TECHNIQUES (FAST)
Département Mathématiques, Informatique et Applications

Licence 3 – MIA – Semestre 6
Année académique 2025–2026

Tableau de bord web pour l'analyse des ventes agricoles

Groupe : 29

Membres :

Bignon Igor Eunérih DADE
Marie Ange Helena Andrea DJEGUEDE
Azondokindé Dieudonné Jules KAKPO
Aina René Régis KIKI

Tuteur : Dr Abdou Wahidi BELLO

Période : Du 20 avril 2026 au 12 juin 2026

Rapport de stage – Session de Juin 2026

Remerciements

Nous tenons à exprimer notre sincère gratitude à toutes les personnes qui ont contribué au bon déroulement de ce stage et à la réalisation de ce projet.

Nous remercions tout d'abord notre tuteur, le Dr Abdou Wahidi Bello, pour son encadrement, ses orientations méthodologiques et sa disponibilité tout au long de la période de stage.

Nous adressons également nos remerciements à l'ensemble du corps enseignant du département de Mathématiques, Informatique et Applications de la Faculté des Sciences et Techniques de l'Université d'Abomey-Calavi, pour la qualité de la formation dispensée en Licence 3 MIA.

Enfin, nous remercions nos familles et proches pour leur soutien constant.

Résumé

Le présent rapport rend compte du travail réalisé dans le cadre du stage de fin de semestre en Licence 3 MIA à la Faculté des Sciences et Techniques de l'Université d'Abomey-Calavi. Le sujet porte sur la conception et la réalisation d'un tableau de bord web destiné à l'analyse des ventes de produits agricoles au Bénin.

La solution développée repose sur une architecture client-serveur à trois couches : un frontend construit avec Next.js 16 et React 19, un backend API développé avec Express 5, et une base de données PostgreSQL hébergée sur Neon. L'authentification est gérée par Better Auth avec un système de cookies HTTP-only sécurisés. Le schéma de données comprend sept modèles dont quatre entités métier (Produit, Client, Vente, User) et trois entités d'authentification (Session, Account, Verification).

L'application offre un tableau de bord interactif avec des indicateurs clés de performance (chiffre d'affaires, nombre de ventes, produit vedette, tendance mensuelle), des graphiques d'évolution temporelle sur 24 mois, un classement des produits et des villes, ainsi qu'un formulaire de saisie de nouvelles ventes. La visualisation des données est assurée par Chart.js. Le jeu de données de démonstration comprend 604 ventes réparties sur 24 mois, 40 clients dans 6 villes, 10 produits agricoles et 5 utilisateurs. L'application est déployée sur Vercel (frontend), Render (backend) et Neon (base de données).

Mots-clés : tableau de bord, ventes agricoles, Next.js, Express, PostgreSQL, Prisma, Chart.js, Better Auth, analyse de données, application web.

Table des matières

Remerciements	i
Résumé	ii
1 Présentation du sujet	1
1.1 Contexte général	1
1.2 Problème traité	1
1.3 Objectifs du stage	2
2 Analyse des besoins	3
2.1 Besoins fonctionnels	3
2.2 Besoins non fonctionnels	3
2.3 Choix technologiques	5
3 Conception	6
3.1 Modèle Conceptuel de Données (MCD)	6
3.2 Modèle Logique de Données (MLD)	7
3.3 Architecture de l'application	7
4 Réalisation	9
4.1 Structure de la base de données	9
4.2 Requêtes principales	10
4.2.1 Indicateurs de synthèse	10
4.2.2 Données des graphiques	11
4.2.3 Création d'une vente	11
4.3 Développement de l'application	12
4.3.1 Authentification et gestion des sessions	12
4.3.2 Tableau de bord et indicateurs clés	12
4.3.3 Visualisation des données	13
4.3.4 Saisie des ventes	13
4.3.5 Filtrage par période	13
4.4 Interfaces utilisateur	14

5	Tests	16
5.1	Stratégie de test	16
5.2	Jeux de tests	16
5.3	Bugs rencontrés et corrections	17
6	Conclusion	19
6.1	Bilan du stage	19
6.2	Difficultés rencontrées	19
6.3	Compétences acquises	20
6.4	Perspectives d'amélioration	20
	Bibliographie	21
A	Script de création de la base de données	22
B	Extraits de code significatifs	25
C	Manuel utilisateur résumé	28

Table des figures

3.1	Modèle Conceptuel de Données du système AGRICO	6
3.2	Architecture technique du système AGRICO	8
4.1	Page de connexion de l'application AGRICO	14
4.2	Tableau de bord principal avec indicateurs et graphiques	15
4.3	Formulaire de saisie d'une nouvelle vente	15

Liste des tableaux

2.1	Technologies retenues et justifications	5
4.1	Produits agricoles et prix unitaires	10
5.1	Jeux de tests fonctionnels	16
C.1	Comptes de démonstration	29

Chapitre 1

Présentation du sujet

1.1 Contexte général

Le secteur agricole représente un pilier fondamental de l'économie béninoise. Les producteurs, coopératives et distributeurs de produits agricoles génèrent quotidiennement un volume important de données transactionnelles. Cependant, l'exploitation de ces données à des fins décisionnelles reste limitée, faute d'outils adaptés et accessibles.

Dans le cadre de la formation en Licence 3 au département MIA de la FAST, le présent projet vise à mettre en pratique les compétences acquises en développement web, en bases de données et en analyse de données. Il s'inscrit dans une démarche professionnalisante qui confronte les étudiants à un problème concret du tissu économique local.

Le domaine d'application choisi, la vente de produits agricoles, présente des caractéristiques particulièrement adaptées à un exercice d'analyse de données : volumes transactionnels significatifs, dimensions multiples (produits, clients, géographie, temporalité) et saisonnalité marquée des activités.

1.2 Problème traité

Les acteurs de la filière agricole au Bénin font face à plusieurs difficultés dans le suivi de leurs activités commerciales :

- L'absence d'outil centralisé de consultation des ventes oblige à des traitements manuels coûteux en temps.
- L'impossibilité de visualiser les tendances temporelles empêche l'anticipation des périodes de forte ou faible demande.
- Le manque d'indicateurs synthétiques rend difficile l'évaluation rapide de la performance commerciale.
- L'absence de segmentation géographique et par produit limite la capacité à orienter les efforts commerciaux.

La problématique centrale se formule ainsi : comment concevoir un outil web ergonomique permettant aux acteurs agricoles de saisir, consulter et analyser leurs données de ventes à travers des indicateurs pertinents et des représentations visuelles adaptées ?

1.3 Objectifs du stage

Objectif général : Concevoir et réaliser un tableau de bord web fonctionnel pour l'analyse des ventes de produits agricoles.

Objectifs spécifiques :

- Concevoir un modèle de données adapté au domaine des ventes agricoles, intégrant les dimensions produit, client, vendeur et géographie.
- Développer une API REST sécurisée exposant les données de ventes avec des capacités de filtrage temporel et par vendeur.
- Calculer et présenter des indicateurs clés de performance : chiffre d'affaires total, nombre de ventes, produit le plus vendu et tendance d'évolution.
- Réaliser des visualisations graphiques interactives : évolution mensuelle sur 24 mois, classement des cinq meilleurs produits, répartition par ville et performance par vendeur.
- Implémenter un formulaire de saisie de nouvelles ventes permettant aux vendeurs d'enregistrer leurs transactions en temps réel.
- Mettre en place un système d'authentification sécurisé avec gestion des rôles (administrateur et vendeur).
- Assurer la responsivité de l'interface et proposer un thème clair/sombre pour le confort d'utilisation.

Chapitre 2

Analyse des besoins

2.1 Besoins fonctionnels

Les besoins fonctionnels identifiés pour le système sont les suivants :

1. **Authentification et gestion des rôles.** Le système doit permettre la connexion des utilisateurs par e-mail et mot de passe. Deux rôles sont définis : administrateur (accès complet) et vendeur (accès restreint à ses propres ventes). La session doit être maintenue de manière sécurisée via des cookies HTTP-only.
2. **Consultation du tableau de bord.** L'utilisateur authentifié doit accéder à une page principale affichant quatre indicateurs clés (chiffre d'affaires total, nombre de ventes, produit vedette, pourcentage d'évolution), un graphique d'évolution temporelle sur 24 mois, un classement des cinq produits les plus vendus, une répartition des ventes par ville, un comparatif des performances par vendeur et un tableau des ventes récentes.
3. **Filtrage temporel.** L'utilisateur doit pouvoir filtrer les indicateurs et graphiques par mois et par année. Les filtres doivent être synchronisés avec l'URL pour permettre le partage de vues spécifiques.
4. **Saisie de ventes.** Les vendeurs doivent pouvoir enregistrer de nouvelles ventes via un formulaire dédié, en sélectionnant le produit, le client et la quantité. Le montant total doit être calculé automatiquement.
5. **Gestion des clients.** Le système doit permettre la consultation de la liste des clients et l'ajout de nouveaux clients avec leurs informations (nom, prénom, ville).

2.2 Besoins non fonctionnels

Performance. Les temps de réponse des requêtes API doivent rester inférieurs à 500 ms. Le chargement initial du tableau de bord doit afficher des indicateurs de chargement (skeletons) pendant la récupération des données.

Sécurité. Les mots de passe doivent être hachés avant stockage. Les sessions doivent utiliser des cookies HTTP-only non accessibles par JavaScript côté client. Les headers HTTP de sécurité doivent être configurés (Content-Security-Policy, X-Frame-Options). La politique CORS doit

restreindre les origines autorisées au seul frontend. Toutes les données en entrée doivent être validées côté serveur par des schémas Zod.

Portabilité. L'interface doit être responsive et adaptée aux écrans desktop, tablette et mobile. Un mode sombre doit être disponible. L'application doit être déployable sur des plateformes cloud (Vercel pour le frontend, Render pour le backend, Neon pour la base de données) avec une configuration reproductible.

2.3 Choix technologiques

TABLE 2.1 – Technologies retenues et justifications

Composant	Technologie	Justification
Framework frontend	Next.js 16 (App Router)	Rendu serveur, routage fichier, middleware intégré, rewrites pour le proxy API
Bibliothèque UI	React 19	Composants réactifs, hooks, Server Components
Langage	TypeScript 5/6	Typage statique, détection d'erreurs à la compilation
Style	Tailwind CSS 4	Classes utilitaires, design responsive mobile-first
Composants UI	shadcn/ui 4.7	Composants accessibles basés sur Radix UI, personnalisables
Graphiques	Chart.js 4.5 + react-chartjs-2	Graphiques performants (canvas), intégration React native
Fetching	TanStack React Query 5	Cache intelligent, revalidation automatique
Formulaires	react-hook-form + Zod 4	Validation déclarative, schémas réutilisables
État URL	nuqs 2.8	Synchronisation filtres-URL, partage de vues
Framework backend	Express 5	Routage explicite, middleware chaînable, écosystème mature
ORM	Prisma 7.8	Schéma déclaratif, migrations automatiques, client typé
Authentification	Better Auth 1.6	Sessions, rôles, cookies HTTP-only natifs
Base de données	PostgreSQL (Neon)	Serverless, scalable, compatible Prisma
Sécurité HTTP	Helmet 8	Configuration automatique des headers de sécurité

Chapitre 3

Conception

3.1 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données identifie les entités du domaine et leurs associations. Le système distingue deux sous-domaines : l'authentification (géré par Better Auth) et le métier (spécifique à l'application). La figure 3.1 présente le diagramme entité-association du système.

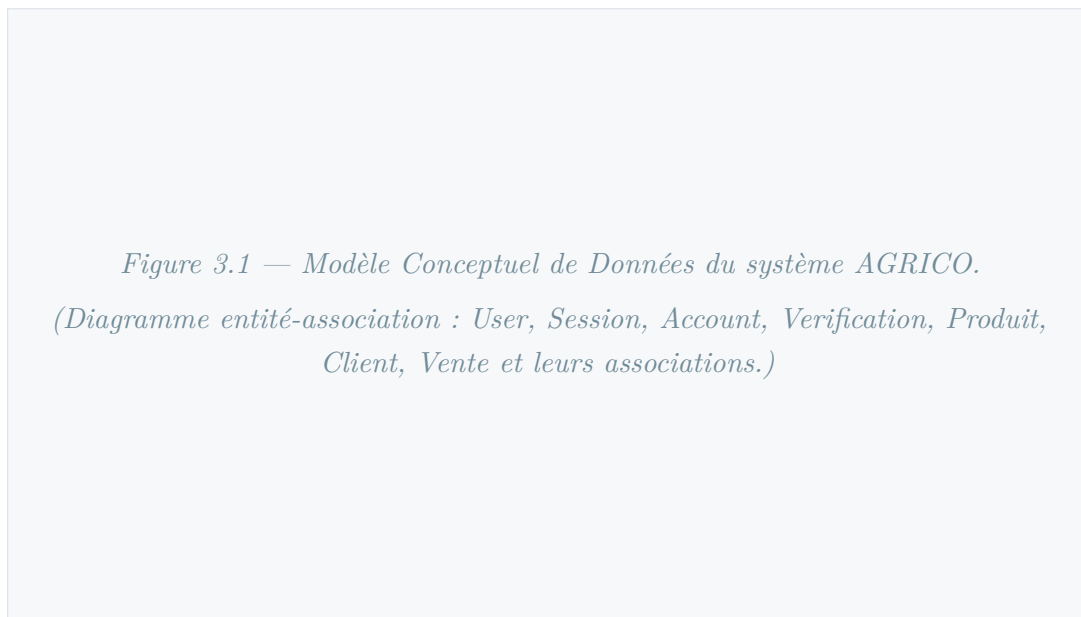


FIGURE 3.1 – Modèle Conceptuel de Données du système AGRICO

Les entités identifiées sont les suivantes :

- **Produit.** Représente un produit agricole commercialisé. Attributs : identifiant unique, nom, catégorie (céréales, tubercules, fruits, légumes, oléagineux, épices) et prix unitaire en francs CFA.
- **Client.** Représente un acheteur de produits agricoles. Attributs : identifiant unique, nom, prénom et ville de résidence. La ville est modélisée comme un attribut simple du client (et non comme une entité séparée), car les besoins d'analyse géographique se limitent à une agrégation par nom de ville.

- **User (Utilisateur).** Représente un utilisateur du système. Attributs : identifiant, e-mail unique, nom, rôle (« admin » ou « vendeur »), indicateur de vérification d’e-mail, image optionnelle, horodatages de création et de mise à jour. Les vendeurs sont des utilisateurs ayant le rôle « vendeur » ; il n’existe pas d’entité Vendeur séparée.
- **Vente.** Représente une transaction commerciale unitaire. Chaque vente associe un produit, un vendeur (utilisateur) et un client, avec une quantité entière et une date de vente. Le montant total est calculé automatiquement (quantité multipliée par le prix unitaire). Une vente concerne un seul produit ; le modèle ne comporte pas de notion de ligne de vente ou de panier multi-produits.
- **Session.** Maintient l’état de connexion d’un utilisateur avec un jeton unique et une date d’expiration.
- **Account.** Stocke les credentials d’un utilisateur pour le fournisseur d’authentification « credential » (e-mail/mot de passe haché).
- **Vérification.** Gère les jetons temporaires pour la vérification d’e-mail et la réinitialisation de mot de passe.

Les associations sont : User – Session (1,N), User – Account (1,N), User – Vente (1,N), Produit – Vente (1,N) et Client – Vente (1,N).

3.2 Modèle Logique de Données (MLD)

Le modèle logique traduit le MCD en tables relationnelles. Les identifiants sont générés automatiquement (cuid pour les entités métier, chaînes pour les entités d’authentification). Les clés étrangères portent le suffixe Id. La notation relationnelle est la suivante :

- User(id, email, name, emailVerified, image, role, createdAt, updatedAt)
- Session(id, expiresAt, token, #userId)
- Account(id, accountId, providerId, #userId, password)
- Verification(id, identifiant, value, expiresAt)
- Produit(id, nom, categorie, prixUnitaire)
- Client(id, nom, prenom, ville)
- Vente(id, #produitId, #vendeurId, #clientId, quantite, dateVente, montantTotal, createdAt)

Les règles d’intégrité sont les suivantes : la suppression d’un utilisateur entraîne la suppression en cascade de ses sessions et comptes ; le montant total d’une vente est calculé à la création ; l’e-mail d’un utilisateur est unique ; le jeton de session est unique. Des index sont définis sur les colonnes dateVente, vendeurId et produitId de la table Vente pour optimiser les requêtes d’agrégation.

3.3 Architecture de l’application

L’architecture du système suit le patron client-serveur à trois niveaux, comme illustré par la figure 3.2.

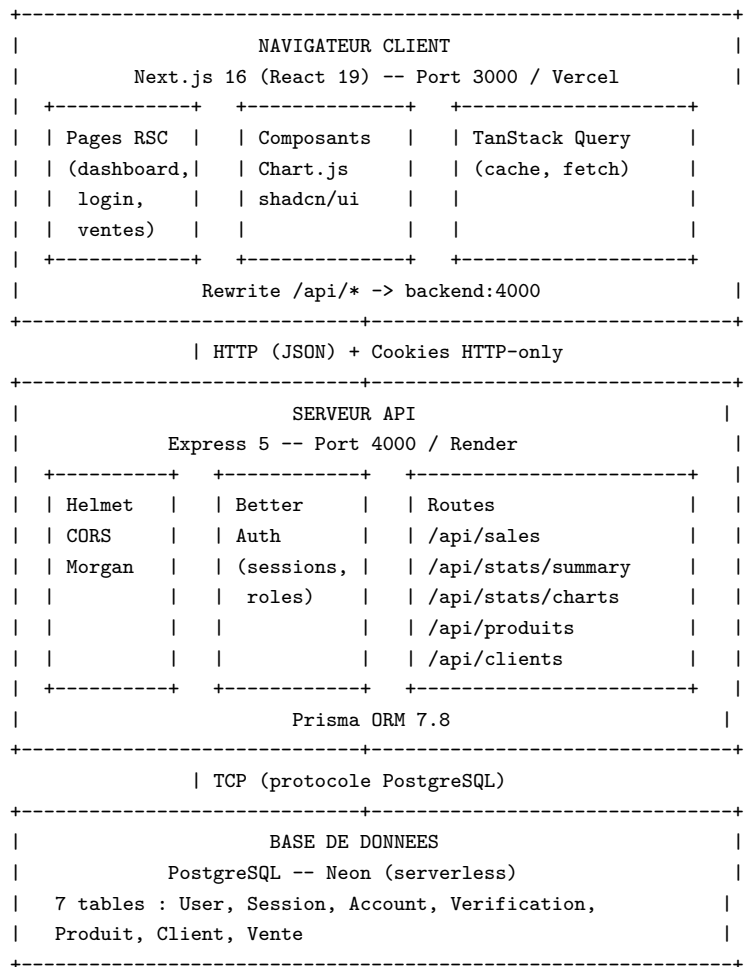


FIGURE 3.2 – Architecture technique du système AGRICO

Le mécanisme de proxy constitue un point d'architecture notable. Les cookies HTTP-only nécessitent que les requêtes d'authentification et les requêtes de données proviennent de la même origine. Next.js est configuré en mode proxy via sa fonctionnalité rewrites : le frontend émet ses requêtes vers `/api/*` (même origine), Next.js les réécrit vers le backend Express, et les cookies `better-auth.session_token` sont ainsi posés et lus sur le domaine du frontend.

Le flux d'une requête typique suit huit étapes :

1. L'utilisateur interagit avec l'interface.
2. TanStack Query déclenche un fetch vers l'API.
3. Le middleware Next.js vérifie la présence du cookie de session.
4. Next.js réécrit la requête vers le backend.
5. Le middleware `requireSession` d'Express valide le jeton.
6. La route appelle Prisma pour agréger les données.
7. Le résultat JSON est retourné.
8. React Query met en cache le résultat et déclenche le rendu.

Chapitre 4

Réalisation

4.1 Structure de la base de données

La base de données PostgreSQL est hébergée sur Neon, une plateforme serverless offrant un plan gratuit adapté au développement. Le schéma est géré par Prisma ORM : les modifications sont versionnées sous forme de migrations SQL générées automatiquement. Le listing 4.1 présente un extrait du schéma Prisma décrivant les entités métier.

```
1 model Produit {
2   id          String  @id @default(cuid())
3   nom         String
4   categorie   String
5   prixUnitaire Float
6   ventes      Vente[]
7 }
8
9 model Client {
10  id      String  @id @default(cuid())
11  nom     String
12  prenom  String
13  ville   String
14  ventes  Vente[]
15 }
16
17 model Vente {
18   id          String  @id @default(cuid())
19   produitId   String
20   vendeurId   String
21   clientId    String
22   quantite    Int
23   dateVente   DateTime
24   montantTotal Float
25   createdAt   DateTime @default(now())
26
27   produit Produit @relation(fields: [produitId], references: [id])
28   vendeur User    @relation(fields: [vendeurId], references: [id])
29   client  Client  @relation(fields: [clientId], references: [id])
30 }
```



```

31  @@index([dateVente])
32  @@index([vendeurId])
33  @@index([produitId])
34  }

```

Listing 4.1 – Schéma Prisma des entités métier (extrait de prisma/schema.prisma)

Le script de seed (`src/seed.ts`) génère un jeu de données représentatif : 5 utilisateurs (1 administrateur, 4 vendeurs), 10 produits agricoles répartis en 6 catégories, 40 clients dans 6 villes (Cotonou, Abomey-Calavi, Porto-Novo, Parakou, Bohicon, Ouidah) et 604 ventes réparties sur 24 mois (juin 2024 à mai 2026) avec une saisonnalité réaliste reflétant les cycles de récolte.

TABLE 4.1 – Produits agricoles et prix unitaires

Produit	Catégorie	Prix unitaire (FCFA)
Maïs	Céréales	250
Riz local	Céréales	450
Manioc	Tubercules	150
Igname	Tubercules	350
Tomate	Légumes	500
Piment	Épices	600
Ananas	Fruits	300
Noix de palme	Oléagineux	200
Soja	Oléagineux	400
Arachide	Oléagineux	280

4.2 Requêtes principales

L'accès aux données est réalisé via le client Prisma, qui génère des requêtes SQL optimisées à partir d'une syntaxe TypeScript déclarative.

4.2.1 Indicateurs de synthèse

Le listing 4.2 présente les requêtes Prisma utilisées pour calculer les indicateurs clés du tableau de bord : chiffre d'affaires total, nombre de ventes et produit le plus vendu pour un mois donné.

```

1  const ventes = await prisma.vente.aggregate({
2    where: {
3      dateVente: { gte: startDate, lt: endDate },
4    },
5    _sum: { montantTotal: true },
6    _count: { id: true },

```

```

7 });
8
9 const topProduit = await prisma.vente.groupBy({
10   by: ['produitId'],
11   where: { dateVente: { gte: startDate, lt: endDate } },
12   _sum: { quantite: true },
13   orderBy: { _sum: { quantite: 'desc' } },
14   take: 1,
15 });

```

Listing 4.2 – Requêtes Prisma pour les indicateurs de synthèse (route GET /api/stats/summary)

4.2.2 Données des graphiques

Le listing 4.3 présente les requêtes Prisma pour l'évolution mensuelle du chiffre d'affaires sur 24 mois et le classement des cinq produits les plus vendus.

```

1 const evolution = await prisma.vente.groupBy({
2   by: ['dateVente'],
3   where: { dateVente: { gte: twentyFourMonthsAgo } },
4   _sum: { montantTotal: true },
5   orderBy: { dateVente: 'asc' },
6 });
7
8 const top5 = await prisma.vente.groupBy({
9   by: ['produitId'],
10  where: { dateVente: { gte: startDate, lt: endDate } },
11  _sum: { quantite: true },
12  orderBy: { _sum: { quantite: 'desc' } },
13  take: 5,
14 });

```

Listing 4.3 – Requêtes Prisma pour les graphiques (route GET /api/stats/charts)

4.2.3 Création d'une vente

Le listing 4.4 présente la logique de création d'une vente, incluant la récupération du prix unitaire du produit et le calcul automatique du montant total.

```

1 const produit = await prisma.produit.findUniqueOrThrow({
2   where: { id: produitId },
3 });
4
5 const vente = await prisma.vente.create({
6   data: {
7     produitId,
8     vendeurId: session.user.id,
9     clientId,
10    quantite,
11    dateVente: new Date(),

```

```

12     montantTotal: quantite * produit.prixUnitaire,
13   },
14 });

```

Listing 4.4 – Création d’une vente (route POST /api/sales)

4.3 Développement de l’application

4.3.1 Authentification et gestion des sessions

L’authentification repose sur Better Auth, configuré avec le fournisseur « credential » (e-mail et mot de passe haché), le plugin « admin » pour la gestion des rôles, et Prisma comme backend de stockage. Les cookies `better-auth.session_token` sont émis avec les attributs `httpOnly`, `secure` et `sameSite: lax`.

Côté backend, le middleware `requireSession` extrait le jeton de session depuis les headers de la requête, valide la session auprès de Better Auth, et attache les objets `user` et `session` à la requête. Le middleware `requireRole(role)` vérifie le rôle de l’utilisateur authentifié. Côté frontend, le client Better Auth (`lib/auth-client.ts`) expose les méthodes `signIn`, `signOut` et `getSession`. Le middleware Next.js (`middleware.ts`) protège les routes en vérifiant la présence du cookie de session et redirige vers `/login` en cas d’absence.

Le listing 4.5 présente le middleware d’authentification côté backend.

```

1 export const requireSession: RequestHandler = async (req, res, next) => {
2   const session = await auth.api.getSession({
3     headers: fromNodeHeaders(req.headers),
4   });
5   if (!session) {
6     res.status(401).json({ error: "Non authentifié" });
7     return;
8   }
9   req.user = session.user;
10  req.session = session.session;
11  next();
12 };

```

Listing 4.5 – Middleware d’authentification Express (`middleware/auth.ts`)

4.3.2 Tableau de bord et indicateurs clés

Le tableau de bord principal (page /) affiche quatre indicateurs clés via le composant `KpiCard`. Chaque carte présente une icône, un titre, une valeur formatée et une tendance en pourcentage par rapport au mois précédent. Les données sont récupérées par le hook `useSummary(month, year)` qui interroge la route GET `/api/stats/summary`.

Le listing 4.6 présente le hook de récupération des indicateurs.

```

1 export function useSummary(month: number | null, year: number | null) {
2   return useQuery({

```

```

3   queryKey: ['summary', month, year],
4   queryFn: () => fetchApi<SummaryResponse>(
5     '/api/stats/summary?month=${month}&year=${year}'
6   ),
7   });
8 }

```

Listing 4.6 – Hook TanStack Query pour les indicateurs (hooks/use-dashboard-data.ts)

4.3.3 Visualisation des données

La visualisation repose sur Chart.js via le wrapper react-chartjs-2. Quatre composants graphiques sont implémentés :

- **EvolutionChart** : graphique en ligne (Line) montrant l'évolution du chiffre d'affaires mensuel sur 24 mois.
- **Top5ProduitsChart** : graphique en barres horizontales (Bar) classant les cinq produits les plus vendus par quantité.
- **VillesChart** : graphique en anneau (Doughnut) montrant la répartition des ventes par ville.
- **VendeursChart** : graphique en barres verticales (Bar) comparant le chiffre d'affaires généré par chaque vendeur.

Les données sont récupérées par le hook `useCharts(month, year)` qui interroge la route GET `/api/stats/charts`. Les composants graphiques sont marqués « use client » car Chart.js utilise l'API Canvas du navigateur.

4.3.4 Saisie des ventes

La page `/ventes` permet aux vendeurs d'enregistrer de nouvelles transactions via le composant **SaleForm**. Ce formulaire utilise react-hook-form avec un résolveur Zod pour la validation. Il comprend trois champs : sélection du produit (liste déroulante alimentée par GET `/api/produits`), sélection du client (liste déroulante alimentée par GET `/api/clients`), et saisie de la quantité (entier positif). Le montant total est calculé et affiché en temps réel. La soumission déclenche un POST `/api/sales` avec notification toast de succès ou d'erreur via Sonner.

4.3.5 Filtrage par période

Les filtres de mois et d'année sont gérés par le composant **MonthYearFilter** et la bibliothèque `nuqs`, qui synchronise les paramètres d'URL avec l'état React. Un changement de filtre met à jour l'URL (ex. : `?month=3&year=2026`), ce qui invalide les clés de requête TanStack Query et déclenche automatiquement un nouveau fetch vers les routes de statistiques. Ce mécanisme autorise le partage de vues filtrées par simple copie de l'URL.

4.4 Interfaces utilisateur

Page de connexion

La page de connexion (/login) présente un formulaire centré sur l'écran avec les champs e-mail et mot de passe. La validation côté client vérifie le format de l'e-mail et la longueur minimale du mot de passe. Après connexion réussie, l'utilisateur est redirigé vers le tableau de bord.

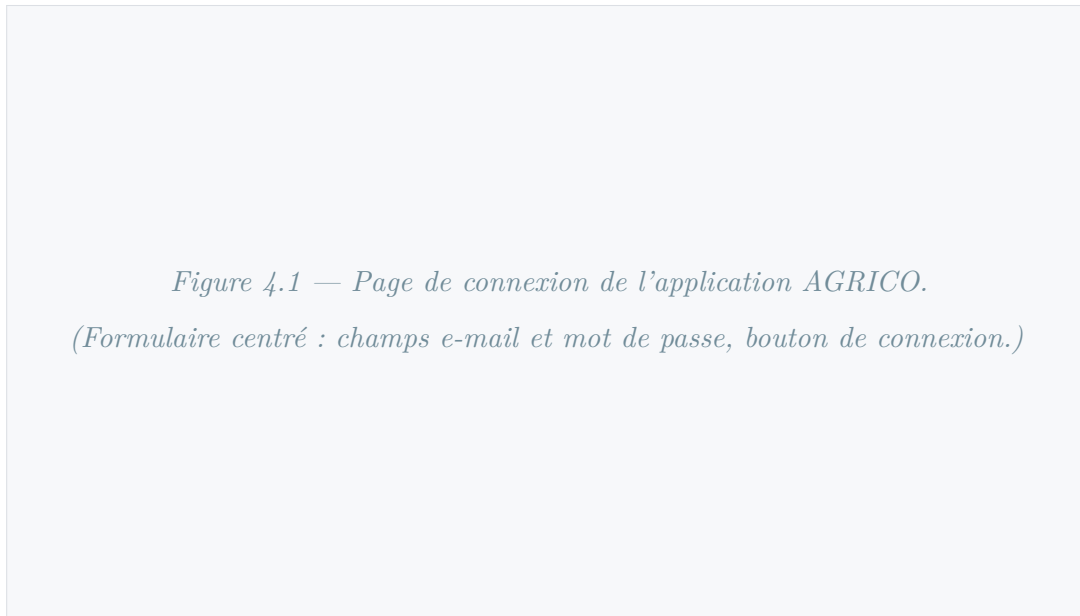


FIGURE 4.1 – Page de connexion de l'application AGRICO

Tableau de bord principal

La page principale (/) s'articule autour de trois zones : une barre latérale permanente regroupant les liens de navigation, les informations utilisateur et le bouton de déconnexion ; une barre supérieure affichant le titre, le sélecteur de thème et les filtres mois/année ; une zone de contenu organisée en grille responsive comprenant quatre cartes KPI alignées en rangée, le graphique d'évolution temporelle sur toute la largeur, le classement des cinq premiers produits et la répartition géographique par ville disposés côte à côte, puis les performances par vendeur et le tableau des ventes récentes. Chaque section affiche un squelette animé pendant le chargement.

*Figure 4.2 — Tableau de bord principal avec indicateurs et graphiques.
(Cartes KPI, graphique d'évolution, top 5 produits, répartition par ville,
performance vendeurs, ventes récentes.)*

FIGURE 4.2 – Tableau de bord principal avec indicateurs et graphiques

Page de saisie des ventes

La page de saisie (/ventes) permet aux vendeurs d'enregistrer de nouvelles transactions. Elle contient un formulaire structuré avec sélecteurs pour le produit et le client, un champ de quantité avec validation en temps réel, l'affichage dynamique du montant total calculé, et un bouton de soumission avec état de chargement.

*Figure 4.3 — Formulaire de saisie d'une nouvelle vente.
(Sélecteurs produit et client, champ quantité, montant total, bouton
d'enregistrement.)*

FIGURE 4.3 – Formulaire de saisie d'une nouvelle vente

Chapitre 5

Tests

5.1 Stratégie de test

La stratégie de test retenue s’articule autour de trois niveaux complémentaires. Le premier repose sur la vérification statique assurée par le typage strict de TypeScript et la validation par Zod, garantissant la cohérence des structures de données tant à la compilation qu’à l’exécution. Le deuxième niveau concerne les tests d’intégration des routes API, qui vérifient le comportement attendu de chaque point de terminaison dans des scénarios représentatifs. Le troisième niveau englobe les tests fonctionnels de bout en bout, couvrant le parcours utilisateur complet depuis l’authentification jusqu’à la consultation du tableau de bord et la saisie d’une vente.

5.2 Jeux de tests

TABLE 5.1: Jeux de tests fonctionnels

N°	Action testée	Données utilisées	Résultat attendu	Obtenu
T1	Connexion valide	e-mail : admin@agrico.bj, mdp : Admin2024!	200, cookie session posé	Conforme
T2	Connexion e-mail inconnu	e-mail : inconnu@test.bj	401, message d’erreur	Conforme
T3	Connexion mot de passe incorrect	e-mail : admin@agrico.bj, mdp : faux	401, message d’erreur	Conforme
T4	Accès route protégée sans session	GET /api/stats/ summary sans cookie	401 Unauthorized	Conforme
T5	Déconnexion	POST /api/auth/ sign-out avec cookie	200, cookie supprimé	Conforme
T6	KPI mois courant	GET /api/stats/ summary?month=5&year=2026	JSON avec totalRevenue > 0	Conforme

Suite page suivante

Suite de la table 5.1

N°	Action testée	Données utilisées	Résultat attendu	Obtenu
T7	KPI mois sans ventes	GET /api/stats/summary?month=12&year=2020	totalRevenue = 0, totalSales = 0	Conforme
T8	Évolution 24 mois	GET /api/stats/charts?month=5&year=2026	Tableau de 24 éléments	Conforme
T9	Top 5 produits	GET /api/stats/charts?month=5&year=2026	5 éléments ordonnés décroissant	Conforme
T10	Répartition par ville	GET /api/stats/charts	Tableau avec les 6 villes	Conforme
T11	Performance vendeurs	GET /api/stats/charts	Tableau de 4 éléments	Conforme
T12	Création vente valide	POST /api/sales {produitId, clientId, quantité : 25}	201, montantTotal calculé	Conforme
T13	Quantité invalide	POST /api/sales {quantité : 0}	400, erreur validation Zod	Conforme
T14	Produit inexistant	POST /api/sales {produitId : « invalide »}	404, produit non trouvé	Conforme
T15	Liste produits	GET /api/produits	200, tableau de 10 produits	Conforme
T16	Création client valide	POST /api/clients {nom, prénom, ville}	201, client créé	Conforme
T17	Création client incomplète	POST /api/clients {nom seulement}	400, validation Zod	Conforme

5.3 Bugs rencontrés et corrections

- **Cookies non posés en production.** La séparation entre frontend (Vercel) et backend (Render) empêchait le partage de cookies *cross-origin*. La correction a consisté à implémenter le proxy via les rewrites de Next.js pour unifier l'origine.
- **Hydration mismatch sur les graphiques.** Chart.js utilise l'API Canvas du navigateur, causant une divergence entre le rendu serveur et client. La correction a consisté à ajouter la directive « use client » sur les composants graphiques et à utiliser le chargement dynamique.
- **Filtres non persistés au rechargement.** L'état des sélecteurs mois/année était local (useState), perdu au rechargement de page. La migration vers nuqs a permis la synchronisation état-URL.
- **Erreur CORS sur les requêtes preflight.** Le middleware CORS n'était pas configuré pour les méthodes OPTIONS avec credentials. La configuration explicite de `credentials: true` et des méthodes autorisées a résolu le problème.
- **Montant total incorrect après modification du prix.** Le montant était recalculé à la

lecture au lieu d'être stocké. La correction a consisté à calculer et stocker le `montantTotal` à la création de la vente.

- **Session expirée sans redirection.** Le middleware Next.js ne détectait pas les sessions expirées. L'ajout d'une vérification `get-session` dans le middleware avec redirection vers `/login` a corrigé le comportement.

Chapitre 6

Conclusion

6.1 Bilan du stage

Le projet « Dashboard AGRICO » a abouti à la réalisation d'un tableau de bord web fonctionnel répondant aux objectifs initiaux. Les principaux livrables réalisés sont :

- Une API REST sécurisée avec 12 endpoints couvrant l'authentification, la consultation et la saisie de données.
- Un tableau de bord interactif avec quatre indicateurs clés et quatre graphiques Chart.js (évolution temporelle, top 5 produits, répartition par ville, performance vendeurs).
- Un formulaire de saisie de ventes avec validation en temps réel et calcul automatique du montant.
- Un jeu de données de démonstration réaliste (604 ventes sur 24 mois, 10 produits, 40 clients, 6 villes).
- Un système d'authentification complet avec gestion des rôles (administrateur et vendeur).
- Une interface responsive avec thème clair/sombre.
- Un déploiement cloud fonctionnel sur trois plateformes (Vercel, Render, Neon).

6.2 Difficultés rencontrées

- **Gestion des cookies *cross-origin*.** La séparation physique entre frontend et backend a imposé la mise en place d'un mécanisme de proxy pour que les cookies HTTP-only fonctionnent correctement. La compréhension des politiques de même origine et de SameSite a nécessité des recherches approfondies.
- **Configuration de Better Auth.** La relative jeunesse de cette bibliothèque a engendré une documentation parfois lacunaire. Son intégration avec Prisma ainsi que la configuration des rôles ont requis une phase d'expérimentation et d'exploration des sources communautaires.
- **Saisonnalité des données de seed.** La génération de 604 entrées de ventes selon une distribution temporelle vraisemblable a nécessité la conception d'un algorithme de pondération prenant en compte les cycles agricoles, notamment les périodes de récolte et de soudure.
- **Optimisation des requêtes d'agrégation.** Les calculs statistiques portant sur 24 mois

et impliquant des regroupements selon de multiples dimensions ont nécessité l'ajout d'index sur la table Vente.

- **Compatibilité Tailwind CSS 4.** La version 4 introduit des changements significatifs (configuration CSS-first, suppression de `tailwind.config.js`) qui ont nécessité une adaptation par rapport aux tutoriels existants.

6.3 Compétences acquises

- Maîtrise de Next.js 16 avec l'App Router (route groups, layouts, middleware, rewrites).
- Utilisation avancée de React 19 (Server Components, hooks personnalisés).
- Intégration de shadcn/ui et Radix UI pour des interfaces accessibles.
- Création de visualisations de données avec Chart.js (graphiques en ligne, barres, anneaux).
- Gestion de l'état serveur avec TanStack React Query (cache, invalidation).
- Validation de formulaires avec react-hook-form et Zod.
- Conception d'API REST avec Express 5 et TypeScript.
- Modélisation de données avec Prisma ORM (schéma déclaratif, migrations, requêtes typées).
- Implémentation d'une authentification sécurisée avec Better Auth.
- Utilisation de PostgreSQL serverless via Neon avec optimisation par indexation.
- Déploiement sur plateformes cloud (Vercel, Render, Neon).
- Travail collaboratif avec gestion de versions sous Git.

6.4 Perspectives d'amélioration

- **Export des données.** Ajout de fonctionnalités d'export en PDF et CSV pour les rapports et tableaux de données.
- **Tableau de bord administrateur.** Création d'une interface d'administration permettant la gestion des utilisateurs, des produits et des clients.
- **Notifications et alertes.** Mise en place d'un système de notifications alertant les utilisateurs en cas d'événements significatifs (seuil de stock, objectif de vente atteint).
- **Analyse prédictive.** Intégration de modèles statistiques simples (moyennes mobiles, régression linéaire) pour la prévision des ventes futures.
- **Application mobile.** Adaptation de l'interface pour une utilisation optimale sur smartphone via une Progressive Web App (PWA).
- **Multi-langues.** Internationalisation de l'interface pour supporter le français et les langues locales.
- **Import de données.** Possibilité d'importer des ventes en masse depuis des fichiers CSV ou Excel.

Bibliographie

- [1] *Next.js Documentation*. Vercel. Disponible sur : <https://nextjs.org/docs> (consulté en mai 2026).
- [2] *React Documentation*. Meta. Disponible sur : <https://react.dev> (consulté en mai 2026).
- [3] *Express.js 5.x Documentation*. OpenJS Foundation. Disponible sur : <https://expressjs.com> (consulté en avril 2026).
- [4] *Prisma Documentation*. Prisma Data, Inc. Disponible sur : <https://www.prisma.io/docs> (consulté en avril 2026).
- [5] *Better Auth Documentation*. Disponible sur : <https://www.better-auth.com/docs> (consulté en mai 2026).
- [6] *Chart.js Documentation*. Disponible sur : <https://www.chartjs.org/docs> (consulté en mai 2026).
- [7] *react-chartjs-2*. Disponible sur : <https://react-chartjs-2.js.org> (consulté en mai 2026).
- [8] *shadcn/ui Documentation*. Disponible sur : <https://ui.shadcn.com> (consulté en mai 2026).
- [9] *Tailwind CSS 4 Documentation*. Tailwind Labs. Disponible sur : <https://tailwindcss.com/docs> (consulté en avril 2026).
- [10] *TanStack React Query Documentation*. Disponible sur : <https://tanstack.com/query> (consulté en mai 2026).
- [11] *Zod Documentation*. Disponible sur : <https://zod.dev> (consulté en mai 2026).
- [12] *nuqs — Type-safe URL query state for Next.js*. Disponible sur : <https://nuqs.47ng.com> (consulté en mai 2026).
- [13] *Neon — Serverless PostgreSQL*. Disponible sur : <https://neon.tech/docs> (consulté en avril 2026).
- [14] *Radix UI Primitives*. Disponible sur : <https://www.radix-ui.com> (consulté en mai 2026).
- [15] *Helmet.js — Security headers for Express*. Disponible sur : <https://helmetjs.github.io> (consulté en mai 2026).
- [16] *react-hook-form Documentation*. Disponible sur : <https://react-hook-form.com> (consulté en mai 2026).
- [17] *Vercel Deployment Documentation*. Disponible sur : <https://vercel.com/docs> (consulté en juin 2026).
- [18] *Render Documentation*. Disponible sur : <https://docs.render.com> (consulté en juin 2026).

Annexe A

Script de création de la base de données

```
1 generator client {
2   provider = "prisma-client-js"
3 }
4
5 datasource db {
6   provider = "postgresql"
7   url      = env("DATABASE_URL")
8 }
9
10 model User {
11   id          String   @id
12   email       String   @unique
13   name        String
14   emailVerified Boolean
15   image       String?
16   role        String   @default("vendeur")
17   createdAt   DateTime
18   updatedAt   DateTime
19   sessions    Session[]
20   accounts    Account[]
21   ventes      Vente[]
22 }
23
24 model Session {
25   id          String   @id
26   expiresAt   DateTime
27   token       String   @unique
28   userId      String
29   user        User     @relation(fields: [userId],
30     references: [id], onDelete: Cascade)
31 }
32
33 model Account {
34   id          String   @id
```

```

35     accountId String
36     providerId String
37     userId     String
38     user       User   @relation(fields: [userId],
39         references: [id], onDelete: Cascade)
40     password   String?
41 }
42
43 model Verification {
44     id          String   @id
45     identifier String
46     value       String
47     expiresAt  DateTime
48 }
49
50 model Produit {
51     id          String   @id @default(cuid())
52     nom         String
53     categorie   String
54     prixUnitaire Float
55     ventes      Vente[]
56 }
57
58 model Client {
59     id          String   @id @default(cuid())
60     nom         String
61     prenom     String
62     ville      String
63     ventes     Vente[]
64 }
65
66 model Vente {
67     id          String   @id @default(cuid())
68     produitId   String
69     vendeurId   String
70     clientId    String
71     quantite    Int
72     dateVente   DateTime
73     montantTotal Float
74     createdAt   DateTime @default(now())
75
76     produit Produit @relation(fields: [produitId],
77         references: [id])
78     vendeur User   @relation(fields: [vendeurId],
79         references: [id])
80     client  Client @relation(fields: [clientId],
81         references: [id])
82
83     @@index([dateVente])
84     @@index([vendeurId])
85     @@index([produitId])
86 }

```

Listing A.1 – Schéma Prisma complet (prisma/schema.prisma)

Annexe B

Extraits de code significatifs

```
1 const API_BASE = process.env.NEXT_PUBLIC_APP_URL || '';
2
3 export async function fetchApi<T>(endpoint: string): Promise<T> {
4   const response = await fetch(`${API_BASE}${endpoint}`, {
5     credentials: 'include',
6     headers: { 'Content-Type': 'application/json' },
7   });
8   if (!response.ok) {
9     throw new Error('Erreur API : ${response.status}');
10  }
11  return response.json();
12 }
```

Listing B.1 – Wrapper API fetch côté frontend (lib/api.ts)

```
1 router.get('/summary', requireSession, async (req, res) => {
2   const month = parseInt(req.query.month as string)
3     || new Date().getMonth() + 1;
4   const year = parseInt(req.query.year as string)
5     || new Date().getFullYear();
6
7   const startDate = new Date(year, month - 1, 1);
8   const endDate = new Date(year, month, 1);
9
10  const ventes = await prisma.vente.aggregate({
11    where: { dateVente: { gte: startDate, lt: endDate } },
12    _sum: { montantTotal: true },
13    _count: { id: true },
14  });
15
16  const topProduit = await prisma.vente.groupBy({
17    by: ['produitId'],
18    where: { dateVente: { gte: startDate, lt: endDate } },
19    _sum: { quantite: true },
20    orderBy: { _sum: { quantite: 'desc' } },
21    take: 1,
22  });
```



```

23
24   res.json({
25     totalRevenue: ventes._sum.montantTotal || 0,
26     totalSales: ventes._count.id,
27     topProduct: topProduit[0] || null,
28   });
29 });

```

Listing B.2 – Route de statistiques résumées (routes/stats.ts, extrait)

```

1  export function useCharts(
2    month: number | null,
3    year: number | null
4  ) {
5    return useQuery({
6      queryKey: ['charts', month, year],
7      queryFn: () => fetchApi<ChartsResponse>(
8        `/api/stats/charts?month=${month}&year=${year}`
9      ),
10   });
11 }
12
13 export function useRecentSales(
14   month: number | null,
15   year: number | null
16 ) {
17   return useQuery({
18     queryKey: ['recent-sales', month, year],
19     queryFn: () => fetchApi<Sale[]>(
20       `/api/sales?month=${month}&year=${year}&limit=10`
21     ),
22   });
23 }

```

Listing B.3 – Hook TanStack Query pour les graphiques (hooks/use-dashboard-data.ts, extrait)

```

1  const saleSchema = z.object({
2    produitId: z.string().min(1, "Sélectionnez un produit"),
3    clientId: z.string().min(1, "Sélectionnez un client"),
4    quantite: z.number().int()
5      .positive("La quantité doit être positive"),
6  });
7
8  export function SaleForm() {
9    const form = useForm<z.infer<typeof saleSchema>>>({
10      resolver: zodResolver(saleSchema),
11    });
12
13    const handleSubmit = async (
14      data: z.infer<typeof saleSchema>
15    ) => {

```

```

16     const response = await fetchApi('/api/sales', {
17         method: 'POST',
18         body: JSON.stringify(data),
19     });
20     toast.success("Vente enregistrée avec succès");
21 };
22
23 return (
24     <form onSubmit={form.handleSubmit(handleSubmit)}>
25         {/* Sélecteurs produit, client, quantité */}
26     </form>
27 );
28 }

```

Listing B.4 – Composant de formulaire de saisie (components/forms/SaleForm.tsx, extrait)

```

1 import { NextRequest, NextResponse } from 'next/server';
2
3 export function middleware(request: NextRequest) {
4     const sessionCookie = request.cookies.get(
5         'better-auth.session_token'
6     );
7
8     if (!sessionCookie) {
9         return NextResponse.redirect(
10             new URL('/login', request.url)
11         );
12     }
13
14     return NextResponse.next();
15 }
16
17 export const config = {
18     matcher: ['/(?!login|_next|favicon.ico).*'],
19 };

```

Listing B.5 – Middleware Next.js de protection des routes (middleware.ts, extrait)

Annexe C

Manuel utilisateur résumé

Installation

Prérequis. Node.js version 20 ou supérieure, un compte Neon pour la base de données PostgreSQL (ou une instance PostgreSQL locale), npm comme gestionnaire de paquets.

1. Cloner le dépôt

```
git clone git@github.com:chrisregis100/dashboard_agrico.git
cd dashboard_agrico
```

2. Configurer le backend

```
cd backend
npm install
```

Créer un fichier `.env` à partir de `.env.example` :

```
DATABASE_URL="postgresql://user:password@host/database"
BETTER_AUTH_SECRET="une-cle-secrete-aleatoire"
BETTER_AUTH_URL="http://localhost:4000"
FRONTEND_URL="http://localhost:3000"
PORT=4000
```

Appliquer les migrations et générer les données de démonstration :

```
npx prisma migrate dev
npm run db:seed
```

Démarrer le serveur :

```
npm run dev
```

Le backend est accessible sur `http://localhost:4000`.

3. Configurer le frontend

```
cd ../frontend
npm install
```

Créer un fichier `.env.local` :

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
BACKEND_URL=http://localhost:4000
```

Démarrer le serveur de développement :

```
npm run dev
```

Le frontend est accessible sur `http://localhost:3000`.

Utilisation

Connexion. Accéder à `http://localhost:3000/login`. Les comptes de démonstration suivants sont disponibles :

TABLE C.1 – Comptes de démonstration

E-mail	Rôle	Mot de passe
admin@agrico.bj	Administrateur	Admin2024!
koffi@agrico.bj	Vendeur	Vendeur2024!
aicha@agrico.bj	Vendeur	Vendeur2024!
yves@agrico.bj	Vendeur	Vendeur2024!
grace@agrico.bj	Vendeur	Vendeur2024!

Navigation. Après connexion, la barre latérale gauche permet d'accéder aux sections Dashboard (indicateurs et graphiques) et Ventes (formulaire de saisie).

Filtrage des données. Les sélecteurs de mois et d'année dans la barre supérieure permettent de filtrer les indicateurs et graphiques. Les filtres sont synchronisés avec l'URL : il est possible de partager un lien pointant vers une vue spécifique.

Saisie d'une vente. Accéder à la page « Ventes », sélectionner un produit et un client dans les listes déroulantes, saisir la quantité souhaitée, vérifier le montant total affiché automatiquement, puis cliquer sur « Enregistrer la vente ». Un message de confirmation apparaît en cas de succès.

Thème. Le bouton de thème dans la barre supérieure permet de basculer entre le mode clair et le mode sombre. Le choix est sauvegardé automatiquement.

Déploiement en production

Base de données (Neon). Créer un projet sur <https://neon.tech>, récupérer la chaîne de connexion PostgreSQL, l'utiliser comme valeur de `DATABASE_URL`, exécuter `npx prisma migrate deploy` puis `npm run db:seed`.

Backend (Render). Connecter le dépôt Git à Render. Le fichier `render.yaml` à la racine configure automatiquement le service. Définir les variables d'environnement (`DATABASE_URL`, `BETTER_AUTH_SECRET`, `BETTER_AUTH_URL`, `FRONTEND_URL`).

Frontend (Vercel). Importer le projet depuis Git sur <https://vercel.com>, sélectionner le dossier `frontend` comme racine du projet, définir les variables d'environnement (`NEXT_PUBLIC_APP_URL`, `BACKEND_URL`).

*Rapport rédigé par le Groupe 29 — Licence 3 MIA — FAST, Université d'Abomey-Calavi —
Année académique 2025–2026*